

Quantum Information Theory

Lesson 2: Efficient Coding

Andrei Tănăsescu



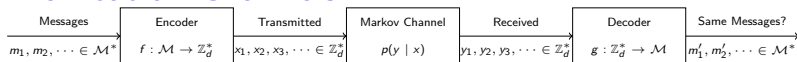
University POLITEHNICA of Bucharest

Feb 26, 2026

Contents

- ▶ Communication Channels
- ▶ Decision Trees
- ▶ Huffman Coding
- ▶ Shannon's Source Coding Theorem
- ▶ Shannon Information Theory

Communication Channels



Let's say we have a memoryless source of random data.

At each time t it will output a random message denoted $m_t \in \mathcal{M}$. Because the generator is memoryless, the messages are independent and identically distributed random variables i.e. $m_t \sim M$ for all t .

Now let's say we want to send these messages over a wire.

We have to encode the messages into a sequence of symbols.

For simplicity of notation we model this as a function $f : \mathcal{M} \rightarrow \mathbb{Z}_d^*$.

At the end, they will need to be decoded using a $g : \mathbb{Z}_d^* \rightarrow \mathcal{M}$.

On the way, the symbols may get corrupted or otherwise changed.

Considering $\mathbf{p} = |i\rangle$ we find classical channels are modelled by stochastic matrices $\mathbf{D} \in \text{Mat}_n([0, 1])$ whose columns sum to 1,

$$\underbrace{\Pr[Y = j]}_{q_j} = \sum_{i=1}^m \underbrace{\Pr[Y = j | X = i]}_{D_{j,i}} \underbrace{\Pr[X = i]}_{p_i}.$$

Decision Trees

Let's say we have an encoding $f : \{A, \dots, F\} \rightarrow \mathbb{Z}_3^*$.

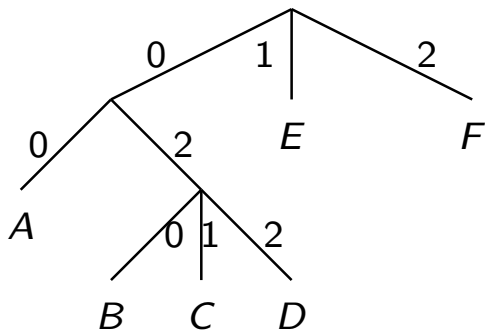
We can represent it as a table or as a decision tree.

The tree shows how to decode f with a finite state translator.

The internal nodes are its states and the edges indicate transitions.

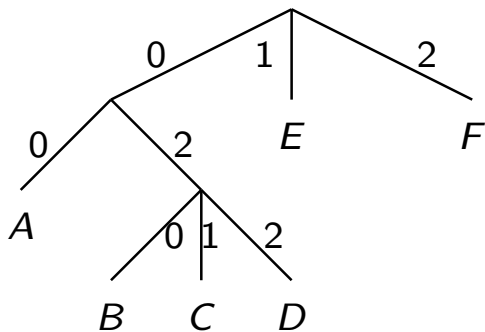
Terminal edges loop back to the initial state and output the leaf.

Message	Code
<i>A</i>	00
<i>B</i>	020
<i>C</i>	021
<i>D</i>	022
<i>E</i>	1
<i>F</i>	2



Decision Trees

Message	Code
<i>A</i>	00
<i>B</i>	020
<i>C</i>	021
<i>D</i>	022
<i>E</i>	1
<i>F</i>	2

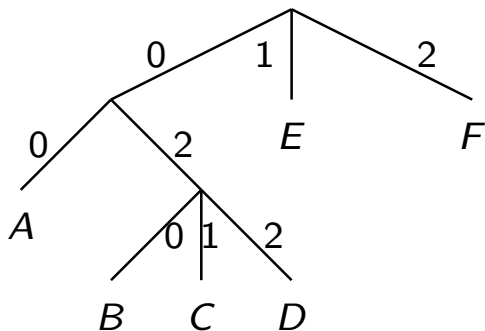


Let's decode the message

02210 00220 20112 02100 21

Decision Trees

Message	Code
<i>A</i>	00
<i>B</i>	020
<i>C</i>	021
<i>D</i>	022
<i>E</i>	1
<i>F</i>	2



Let's decode the message

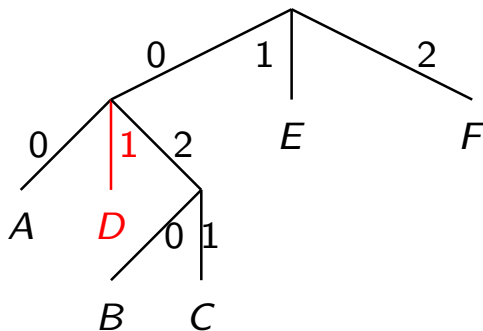
$\underbrace{022}_D \mid \underbrace{1}_E \mid \underbrace{00}_A \mid \underbrace{022}_D \mid \underbrace{020}_B \mid \underbrace{1}_E \mid \underbrace{1}_E \mid \underbrace{2}_F \mid \underbrace{021}_C \mid \underbrace{00}_A \mid \underbrace{2}_F \mid \underbrace{1}_E$

We sent 12 letters as 22 trits, so average 1.8(3) trits/letter.

Can we do better?

Decision Trees

Message	Code
A	00
<i>D</i>	<i>01</i>
B	020
C	021
E	1
F	2



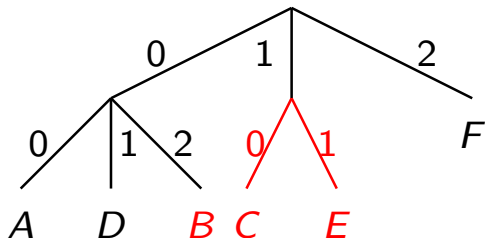
Some changes are always good. Here we freely shortened *D*.

$\underbrace{01}_D$ | $\underbrace{1}_E$ | $\underbrace{00}_A$ | $\underbrace{01}_D$ | $\underbrace{020}_B$ | $\underbrace{1}_E$ | $\underbrace{1}_E$ | $\underbrace{2}_F$ | $\underbrace{021}_C$ | $\underbrace{00}_A$ | $\underbrace{2}_F$ | $\underbrace{1}_E$

We sent 12 letters as 20 trits, so average 1.(6) trits/letter.
Can we do better?

Decision Trees

Message	Code
A	00
D	01
<i>B</i>	<i>02</i>
<i>C</i>	<i>10</i>
<i>E</i>	<i>11</i>
F	2



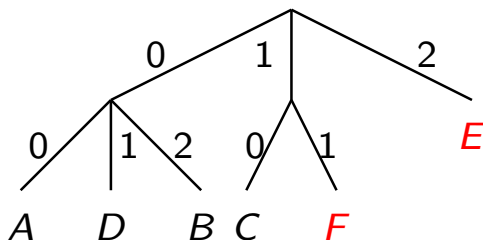
You would think improving *B*, *C* at the cost of *E* would be good.
Problem is we improved two rare letters but broke a common one.

$\underbrace{01}_D \mid \underbrace{11}_E \mid \underbrace{00}_A \mid \underbrace{01}_D \mid \underbrace{02}_B \mid \underbrace{11}_E \mid \underbrace{11}_E \mid \underbrace{2}_F \mid \underbrace{10}_C \mid \underbrace{00}_A \mid \underbrace{2}_F \mid \underbrace{11}_E$

We sent 12 letters as 22 trits, so average 1.8(3) trits/letter.
Can we do better?

Decision Trees

Message	Code
A	00
D	01
B	02
C	10
<i>F</i>	<i>11</i>
<i>E</i>	<i>2</i>



You would think improving *B*, *C* at the cost of *F* would be good.
Problem is *F* occurs just as often as *B*, *C* combined.

01 | 2 | 00 | 01 | 02 | 2 | 2 | 11 | 10 | 00 | 11 | 2
D E A D B E E F C A F E

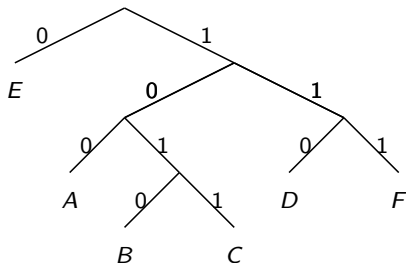
We sent 12 letters as 20 trits, so average 1.(6) trits/letter.
So, any algorithm for doing this thing optimally?

Huffman (Binary) Coding

For the $d = 2$ (bits) case, we can make the optimal decision tree. We will keep merging the lowest frequency nodes, as follows.

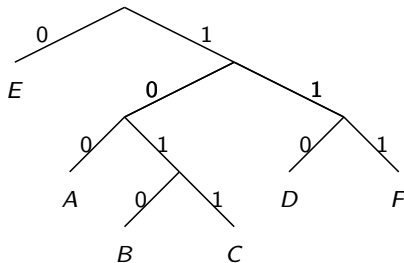
1. $B : 1/12, C : 1/12, A : 2/12, D : 2/12, F : 2/12, E : 4/12$
2. $A : 2/12, D : 2/12, F : 2/12, E : 4/12 \mid BC : 2/12$
3. $D : 2/12, F : 2/12, E : 4/12 \mid ABC : 4/12$
4. $E : 4/12 \mid ABC : 4/12, DF : 4/12$
5. $E : 4/12 \mid ABCDF : 8/12$

Message	Code
A	100
B	1010
C	1011
D	110
E	0
F	111



Huffman (Binary) Coding

Message	Code
A	100
B	1010
C	1011
D	110
E	0
F	111



$\underbrace{110}_D \mid \underbrace{0}_E \mid \underbrace{100}_A \mid \underbrace{110}_D \mid \underbrace{1010}_B \mid \underbrace{0}_E \mid \underbrace{0}_E \mid \underbrace{111}_F \mid \underbrace{1011}_C \mid \underbrace{100}_A \mid \underbrace{111}_F \mid \underbrace{0}_E$

We sent 12 letters as 30 bits, so average 2.5 bits/letter.

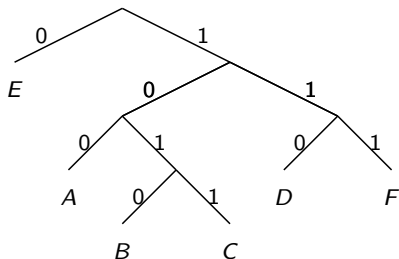
Note: Don't compare 2.5 bits with 1.(6) trits.

Or, if you do, use $1 \text{ trit} = \log_3(2) \text{ bits} \approx 1.584$.

Using that conversion we previously had ≈ 2.641 bits/letter.

Huffman (Binary) Coding

Message m	Prob p_m	Code c_m
A	$2/12$	100
B	$1/12$	1010
C	$1/12$	1011
D	$2/12$	110
E	$4/12$	0
F	$2/12$	111



Let's say we will be sending n letters with this distribution.
We can calculate the expected code length,

$$\begin{aligned}\mathbb{E}[L_n] &= n \sum_{m \in \mathcal{M}} p_m |c_m| = \frac{n}{12} (2 \times 3 + 1 \times 4 + 1 \times 4 + 2 \times 3 + 4 \times 1 \\ &\quad + 2 \times 3) = \frac{30n}{12} = 2.5n.\end{aligned}$$

Note that we did not need the text, only the distribution.

Huffman (Binary) Coding

Let's say we are sending n letters with distribution

$$M \sim \begin{pmatrix} A & B & C & D & E & F \\ 2/12 & 1/12 & 1/12 & 2/12 & 4/12 & 2/12 \end{pmatrix},$$

and let's see what happens if we group these letters in groups of k .

We get a distribution over 6^k values.

We will need to sort that.

Then we run the Huffman algorithm.

Sometimes the efficiency goes up,
sometimes it goes down. You see a
similar effect when upping the
dictionary size in ZIP compression.

But there is a lower bound

$$\mathbb{E}[L_k]/k \geq H(M) = - \sum_{m \in \mathcal{M}} p_m \log_2 p_m,$$

k	$\mathbb{E}[L_k]/k$
1	2.5
2	2.4444444444444446
3	2.4382716049382718
4	2.4382716049382718
5	2.4226337448559674
6	2.432556012802926
7	2.4229211574890583
8	2.4272595640908396
9	2.426134792008897
∞	2.418295834054489

which Huffman coding **does** reach.

Huffman (Binary) Coding

Now let's look at the distribution

$$M \sim \begin{pmatrix} A & B & C \\ 1/7 & 2/7 & 4/7 \end{pmatrix}.$$

A pattern for the up-and-down is that bulking always helps so the efficiency for $k_1 k_2$ is always better than for k_1 and k_2 .

The lower bound here is

$$\begin{aligned} H(M) &= -\frac{1}{7} \log_2 \frac{1}{7} - \frac{2}{7} \log_2 \frac{2}{7} - \frac{4}{7} \log_2 \frac{4}{7} \\ &= \log_2 7 - \frac{1 \log_2 1 + 2 \log_2 2 + 4 \log_2 4}{7} \\ &= \log_2 7 - \frac{10}{7} \approx 1.3787834934861756. \end{aligned}$$

k	$\mathbb{E}[L_k]/k$
1	1.4285714285714286
2	1.4183673469387754
3	1.4071914480077747
4	1.3949396084964598
5	1.3814958053192121
6	1.385746301852686
7	1.3895357359256635
8	1.3895357359256633
9	1.3866485480605377
10	1.3813691188214512
11	1.3817872973469505
12	1.3846615223906829
13	1.3853886076083812
14	1.3842024873744923
15	1.381244557961363
16	1.3802585642139955
∞	1.3787834934861756

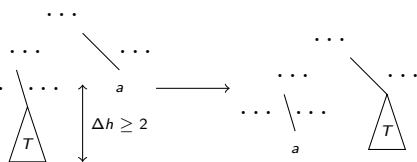
Shannon Source Coding Theorem

Let's say we have a memoryless information source outputting samples of a random variable X . **Shannon's source coding theorem** states that the optimal code satisfies

$$\lim_{n \rightarrow \infty} \frac{1}{n} |\mathbb{E}[L] - nH(X)| = 0,$$

or, using $o(n)$ notation, $nH(X) + o(n) \geq \mathbb{E}[L] \geq nH(X) - o(n)$.

For uniform distributions, if the leaves of the optimal tree have height difference $\Delta h \geq 2$, we can switch the high symbol with a lower subtree T . $\mathbb{E}[L]$ increases by $(\Delta h - 1) \times \frac{1}{N}$ and decreases by $(\Delta h - 1) \times \frac{\text{span}(T)}{N}$ i.e. more.



So the optimal tree is (almost) balanced and complete i.e.

$$\mathbb{E}[L_n] = f \lceil \log_d N^n \rceil + (1 - f) \lfloor \log_d N^n \rfloor = nH(X) \pm 1.$$

Shannon Source Coding Theorem

Let's say now that we know we are sending n symbols with the histogram $\#(x_1) = n_1, \#(x_2) = n_2, \dots, \#(x_N) = n_N$.

Any sequence with this histogram is equally likely to appear.

And there are $\frac{n!}{n_1! \dots n_N!}$ such sequences. It follows that

$$\begin{aligned}\mathbb{E}[\mathbf{L}_n \mid \#(x_1) = n_1, \dots, \#(x_N) = n_N] &\geq \log_d \frac{n!}{n_1! \dots n_N!} - 1 = \log_d n! - \sum_{i=1}^N \log_d n_i! - 1 \\ &= n \log_d n - n \log_d e + o(n) - \sum_{i=1}^N (n_i \log_d n_i - n_i \log_d e + o(n_i)) = - \sum_{i=1}^N n_i \log_d \frac{n_i}{n} + o(n),\end{aligned}$$

where we used the **Stirling approximation**

$$\begin{aligned}\sqrt{2\pi n} \left(\frac{n}{e}\right)^n e^{\frac{1}{12n+1}} &< n! < \sqrt{2\pi n} \left(\frac{n}{e}\right)^n e^{\frac{1}{12n}} \\ \Leftrightarrow \frac{1}{2} \log_d(2\pi) + \frac{1}{2} \log_d n + n \log_d n - n \log_d e + \frac{1}{12n+1} &< \log_d n! \\ &< \frac{1}{2} \log_d(2\pi) + \frac{1}{2} \log_d n + n \log_d n - n \log_d e + \frac{1}{12n} \\ \Rightarrow \log_d n! &= n \log_d n - n \log_d e + o(n).\end{aligned}$$

Shannon Source Coding Theorem

To go further we need to know the distribution of the n_i values in

$$\mathbb{E}[\mathbf{L}_n \mid \#(x_1) = n_1, \dots, \#(x_N) = n_N] \geq - \sum_{i=1}^N n_i \log_d \frac{n_i}{n} + o(n).$$

The number of x_i s drawn from a sample size of n with draw probability p_i is a binomial distribution with mean $\mu = np_i$ and variance $\sigma^2 = np_i(1 - p_i)$. By the **Chebyshev inequality**,

$$\Pr[|n_i - np_i| \geq k\sigma] \leq 1/k^2 \Rightarrow \Pr[|n_i - np_i| \geq \epsilon n] \leq \frac{p_i(1 - p_i)}{\epsilon^2 n}.$$

What's the chance that **any** of the n_i is ϵn away from its mean?

Since $\Pr[A \vee B] = \Pr[A] + \Pr[B] - \Pr[A \wedge B] \leq \Pr[A] + \Pr[B]$,

$$\Pr\left[\bigvee_i |n_i - np_{x_i}| \geq \epsilon n\right] \leq \sum_i \frac{p_{x_i}(1 - p_{x_i})}{\epsilon^2 n} \leq \frac{1}{\epsilon^2 n}.$$

Shannon Source Coding Theorem

Now, with probability $1 - \frac{1}{\epsilon^2 n}$ we know $p - \epsilon \leq n_i/n \leq p + \epsilon$.

We call sequences for which this happens ϵ -typical. In this case,

$$\begin{aligned}\mathbb{E}[L_n] &= \Pr[\epsilon\text{-typical}] \mathbb{E}[L_n \mid \epsilon\text{-typical}] + \Pr[\epsilon\text{-atypical}] \mathbb{E}[L_n \mid \epsilon\text{-atypical}] \\ &\geq \left(1 - \frac{1}{\epsilon^2 n}\right) \min \left\{ -n \sum_{i=1}^N \frac{n_i}{n} \log_d \frac{n_i}{n} + o(n) \mid \epsilon\text{-typical } \{n_i\} \right\} + 0 \\ &\geq \left(1 - \frac{1}{\epsilon^2 n}\right) \left(-n \sum_{i=1}^N (p_i - \epsilon) \log_d (p_i + \epsilon) \right) + o(n),\end{aligned}$$

and taking arbitrarily small ϵ we get $\mathbb{E}[L_n] \geq nH(X) + o(n)$.

Note that we only proved the lower bound. To get an upper bound we need to collect the slack along the way.

Shannon Source Coding Theorem

To get an upper bound, we will assume we tolerate fixed probability π of error, from ϵ -atypical sequences. We want to have $\pi \geq \frac{1}{\epsilon^2 n}$ for arbitrarily small ϵ so we need to consider inputs of length $n \geq \frac{1}{\pi \epsilon^2}$.

So, we have to tell the decoder whether the sequence was ϵ -typical. We also have to send the index of the histogram, i.e. $\mathcal{O}(N \log n)$. Next, we have to consider the other side of the bound above

$$\mathbb{E}[L_n \mid \#, \epsilon\text{-typical}] \leq -n \sum_{i=1}^N (p_i + \epsilon) \log_d(p_i - \epsilon) + o(n),$$

and for each term in the sum we use the 2nd order **Taylor Series**,

$$\begin{aligned} -(p_i + \epsilon) \log_d(p_i - \epsilon) &= -p_i \log_d p_i + \frac{1 - \ln p_i}{\ln d} \epsilon + \frac{3p_i - \xi_i}{2(p_i - \xi_i)^2 \ln d} \epsilon^2 \\ &\stackrel{\xi_i < \epsilon < p_i/2}{\leq} -p_i \log_d p_i + \epsilon \left(\frac{1 - \ln p_i}{\ln d} + \frac{5}{p_i \ln d} \epsilon \right) \end{aligned}$$

$$\mathbb{E}[L_n \mid \#, \epsilon\text{-typical}] \leq n(H(X) + \epsilon C_{X,d}) + o(n).$$

Shannon Source Coding Theorem

Let's say we have a memoryless information source outputting samples of a random variable X . **Shannon's source coding theorem** states that the optimal code satisfies

$$\lim_{n \rightarrow \infty} \frac{1}{n} |\mathbb{E}[L] - nH(X)| = 0,$$

or, using $o(n)$ notation, $nH(X) + o(n) \geq \mathbb{E}[L] \geq nH(X) - o(n)$.

More precisely, we find that **every** code needs at least $H(X)$ dits per letter, and that for any tolerable error rate π and any tolerable code length increase δ there is an $\epsilon \leq \delta/C_{X,d}$ and a minimal packed amount $n \geq 1/(\pi\epsilon^2)$ of letters for which we showed there is a code with good enough code length and probability of decoding.

Information theory research derives relations between quantifiers you need and quantifiers you can measure.

Shannon Information Theory

Entropy is one of those quantifiers that just keeps appearing. We're now going to take a closer look at its story.

One of the problems with probability is that it is multiplicative. For independent X, Y , $\Pr[X = x, Y = y] = \Pr[X = x] \Pr[Y = y]$. When X and Y can take many values, the probabilities on the right are already small; but the one on the left is even smaller. Worse yet, when calculating $\mathbb{E}[XY]$ for say covariance we have a long sum of products between very big and very small numbers. This is horrible for arithmetic error.

Beyond problems, the key insight behind the solution was that information brought by independent events should be additive. Shannon thus wanted a function f such that $f(pq) = f(p) + f(q)$. The only continuous and positive f is $I(x) = -\log_d(x)$ for $d \geq 1$. This function is information content, and its unit depends on d .

IEEE754 Floating Point Arithmetic

```
var S0 float64 = 1e+10
var T float64 = 1e-6
var N int = 10 * 1000

var S float64 = S0 + (float64)(N)*T
var S2 float64 = S0
for idx := 0; idx < N; idx++ {
    S2 += T
}
err := S2 - S
fmt.Println(err) // -0.0095...
```

IEEE754 Floating Point Arithmetic

```
var S0 float64 = 1e+16
var T float64 = 1
var N int = 10 * 1000

var S float64 = S0 + (float64)(N)*T
var S2 float64 = S0
for idx := 0; idx < N; idx++ {
    S2 += T
}
err := S2 - S
fmt.Println(err) // -10000
```

IEEE754 Floating Point Arithmetic

```
var S0 int64 = 1e+16
var T int64 = 1
var N int = 10 * 1000

var S int64 = S0 + (int64)(N)*T
var S2 int64 = S0
for idx := 0; idx < N; idx++ {
    S2 += T
}
err := S2 - S
fmt.Println(err) // 0
```


Shannon Information Theory

The information content associated with an event $A \subseteq \Omega$ is

$$I(A) = -\log_d \Pr[A]$$

so, a sample of the random variable X , brings average information

$$\begin{aligned} H(X) = \mathbb{E}[I(X)] &= \sum_x I(X=x) \Pr[X=x] \\ &= - \sum_x \Pr[X=x] \log_d \Pr[X=x], \end{aligned}$$

a quantity that we denote by $H(X)$ and has the same formula that was already used in physics at the time under the name *entropy*.

Shannon Information Theory

Something to note is that entropy is a function of the probability vector, so the values of X don't matter.

In particular, for an uniform distribution with d entries:

$$H(X) = - \sum_x \frac{1}{d} \log_d \frac{1}{d} = -d \frac{1}{d} \log_d \frac{1}{d} = 1.$$

Now, note that X represents an uniformly random d -ary digit.

For this reason, the unit of entropy with base d is called a **dit**.

If $d=2$, this is called a **bit** for binary digit.

For $d = 3, 4, 5$ the names are trit, quart, quint.

For $d = e$ the name is **nat** for natural.

Shannon Information Theory

Now, let's look at independent events

$$\begin{aligned}H(X, Y) &= - \sum_{x,y} \Pr[X = x, Y = y] \log_d \Pr[X = x, Y = y] \\&= - \sum_{x,y} \Pr[X = x] \Pr[Y = y] \log_d \Pr[X = x] \Pr[Y = y] \\&= - \sum_{x,y} \Pr[X = x] \Pr[Y = y] \log_d \Pr[X = x] \\&\quad - \sum_{x,y} \Pr[X = x] \Pr[Y = y] \log_d \Pr[Y = y] \\&= - \sum_x \Pr[X = x] \log_d \Pr[X = x] - \sum_y \Pr[Y = y] \log_d \Pr[Y = y] \\&= H(X) + H(Y),\end{aligned}$$

so if we have n copies of X we find $H(X^{\otimes n}) = nH(X)$.

The notation assumes that $X^{\otimes n}$ works as if it were a vector,

$$\Pr[X^{\otimes n} = (x_1, \dots, x_n)] = \prod_{i=1}^n \Pr[X = x_i].$$

Shannon Information Theory

If the events X, Y are not independent the above may not hold. Instead, fix $X = x$ and see the information we need to encode Y

$$\begin{aligned}H(Y | X = x) &= \mathbb{E}[I(Y | X = x)] \\&= - \sum_y \Pr[Y = y | X = x] \log_d \Pr[Y = y | X = x] \\&= - \sum_y \frac{\Pr[Y = y, X = x]}{\Pr[X = x]} \log_d \frac{\Pr[Y = y, X = x]}{\Pr[X = x]}\end{aligned}$$

so on average if we are given a random X we need

$$\begin{aligned}H(Y | X) &= \mathbb{E}[H(Y | X = x)] \\&= \sum_x \Pr[X = x] \mathbb{E}[H(Y | X = x)] \\&= - \sum_{x,y} \Pr[Y = y, X = x] \log_d \frac{\Pr[Y = y, X = x]}{\Pr[X = x]} \\&= H(X, Y) - H(X),\end{aligned}$$

a quantity which we call conditional entropy.

Shannon Information Theory

Now let's go back to the coding application and think about what happens if we use an encoder trained for X to encode Y s which have the same values x but different probabilities q_x .

When we were encoding sequences having Np_x of each x the length of the coded message was $-N \sum_i p_x \log p_x$. So, each x added $-\log p_x$ to the length. Now, since the x actually arrive with probability q_x on average each sent character adds

$$-\sum_x q_x \log p_x,$$

a quantity known as cross-entropy.

Since this is a sub-optimal encoding of Y we find that the slack,

$$D_{KL}(Y||X) = -\sum_x q_x \log p_x - H(Y) = -\sum_x q_x \log \frac{p_x}{q_x} \geq 0,$$

a quantity known as relative entropy, is positive.

Shannon Information Theory

D_{KL} is also known as the Kullback-Leibler divergence. It is a measure of distance between two distributions. However, it is not symmetric, like a distance should be.

Now, let's say we want to send pairs (X, Y) of correlated variables. If their probabilistic vectors are $\mathbf{p}_X, \mathbf{p}_Y$ we know that $\mathbf{p}_{X,Y}$ may or may not equal $\mathbf{p}_X \otimes \mathbf{p}_Y$. Considering that X and Y may have many possible values, if we are estimating the densities, it is much easier to estimate the marginals than the joint distribution. So, it makes sense to ask how inefficient it is to code them as independent,

$$I(X : Y) = D_{KL}(\mathbf{p}_{X,Y} \| \mathbf{p}_X \otimes \mathbf{p}_Y),$$

a quantity known as mutual information.

Shannon Information Theory

There are multiple formulas for mutual information, e.g.

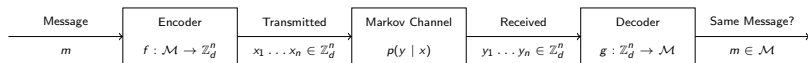
$$\begin{aligned} I(X : Y) &= D_{KL}(\mathbf{p}_{X,Y} \| \mathbf{p}_X \otimes \mathbf{p}_Y) \\ &= - \sum_{x,y} p_{x,y} \log \frac{p_x p_y}{p_{x,y}} \\ &= - \sum_{x,y} p_{x,y} \log p_x - \sum_{x,y} p_{x,y} \log p_y + \sum_{x,y} p_{x,y} \log p_{x,y} \\ &= H(X) + H(Y) - H(X, Y) \\ &= H(X) - H(X | Y) = H(Y) - H(Y | X) \\ &= H(X, Y) - H(X | Y) - H(Y | X), \end{aligned}$$

but the fact that $D_{KL}(\mathbf{p} \| \mathbf{q}) \geq 0$ shows us that $H(X | Y) \leq H(X)$ and $H(X, Y) \leq H(X) + H(Y)$ w/ equality iff. X, Y are independent;

Similarly: if $Y = f(X)$ for a deterministic f , then we get

$$H(f(X)) = H(Y) = H(X) + \underbrace{H(Y | X)}_{H(f(X)|X)=0} - H(X | Y) \leq H(X).$$

Shannon Information Theory



This may all have seemed a bit too much.

However, every quantity before has its applications.

For example, while the Shannon source coding theorem considers a noiseless (identity) channel, we may consider what happens when the Markov channel (noise) is actually in play (but we fix n).

In this scenario, the encoder produces sequences of variables X . The noise then corrupts them into Y , obtained from X using $\mathbf{D}$. The more noise, the lower $I(X : Y)$, i.e. at max noise Y is independent from X . Of course, this depends on X .

The capacity of a channel is defined as $C = \sup_X I(X : Y)$.

The noisy channel coding theorem states that, for any tolerable error rate ϵ , the maximal communication rate ($H(X)$) with maximal error rate ϵ is C . Above C , as $n \rightarrow \infty$ errors become irrecoverable.

Follow-up Reading

- ▶ M.Goemans – Principles of Discrete Applied Mathematics
MIT OCW (PDF), Shannon's Noiseless Coding Theorem
- ▶ T.M.Cover, J.A.Thomas – Elements of Information Theory
John Wiley and Sons (2012), Ch.2-Entropy,5-Codes,6-Horses